

SIMPLEMESSAGEBUS · PHASE 1

# Internal Message Broker

User guide for publishers and subscribers

---

How to connect an application to the broker, publish messages, and receive them reliably by webhook, SignalR or pull, plus what the broker administrator does to set you up.

Version 1.0 · October 2026

API version: `/api/v1` · Client library: `MessageBroker.Contracts`

# Contents

---

## 1. Overview

- 1.1 What the broker does
- 1.2 Key concepts
- 1.3 Who does what
- 1.4 Delivery guarantees

## 2. Getting connected

- 2.1 What to ask the administrator for
- 2.2 What the administrator does
- 2.3 Authenticating your calls

## 3. Publisher guide

- 3.1 Publishing a message
- 3.2 Idempotency and retries
- 3.3 The outbox pattern
- 3.4 Message design
- 3.5 Tracking what you sent
- 3.6 Publisher checklist

## 4. Subscriber guide

- 4.1 Choosing a delivery mode
- 4.2 Rules every subscriber follows
- 4.3 The delivery format
- 4.4 Settling: ACK, NACK, dead-letter, renew
- 4.5 Retries and backoff

## 5. Webhook subscribers

## 6. SignalR subscribers

## 7. Pull subscribers

## 8. Dead letters and troubleshooting

- 8.1 Reading your dead-letter queue
- 8.2 Getting a message requeued
- 8.3 Errors and what to do

## 9. Reference

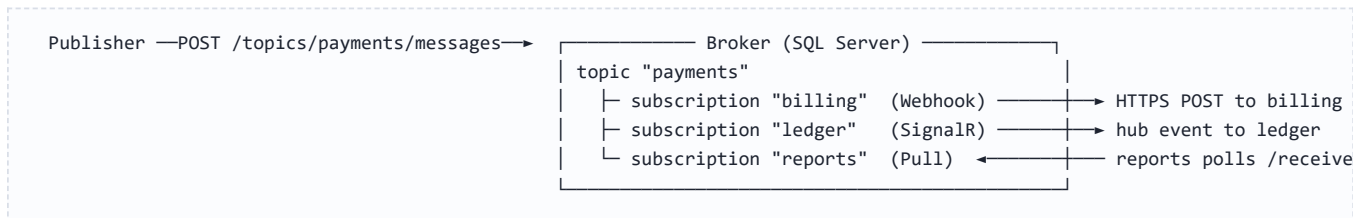
- 9.1 Subscription settings
- 9.2 Limits
- 9.3 Endpoints
- 9.4 Subscriber checklist
- 9.5 Glossary

# 1. Overview

## 1.1 What the broker does

The Internal Message Broker moves messages between internal applications. A **publisher** sends a message to a named **topic**. Every **subscription** on that topic gets its own copy, a **delivery**, and the broker hands each delivery to its subscriber. It keeps retrying until the subscriber confirms the work is done, or until the delivery is moved to a dead-letter queue for a person to look at.

Publishers and subscribers never call each other directly. A publisher does not need to know who listens, and a subscriber that is down or slow does not slow the publisher down: its messages wait in the broker.



## 1.2 Key concepts

| Term                    | Meaning                                                                                                                                                                                             |
|-------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Application             | A registered client (a service, job or app). It authenticates with an <b>API key</b> , and it can hold two active keys so keys can be rotated without downtime.                                     |
| Topic                   | A named channel such as <code>payments</code> . Publishers need <b>Publish</b> permission on it.                                                                                                    |
| Subscription            | A named consumer of one topic, owned by one application, with one <b>delivery mode</b> (Webhook, SignalR or Pull) and its own retry settings.                                                       |
| Message                 | What a publisher sends: a <code>messageType</code> , an optional <code>correlationId</code> , optional string <code>properties</code> , and a JSON <code>payload</code> . A message is stored once. |
| Delivery                | One message for one subscription. Each subscription progresses, retries and fails independently.                                                                                                    |
| Lease and lock token    | When a delivery is handed out it is <b>leased</b> until <code>lockedUntil</code> , and comes with a <code>lockToken</code> . Only that token can settle it, and only before the lease ends.         |
| Attempt                 | One try at a delivery. Every attempt is recorded with its outcome, error code and duration.                                                                                                         |
| Dead-letter queue (DLQ) | Where a delivery goes when it cannot succeed: retries ran out, the subscriber rejected it, or its time-to-live passed. An administrator can requeue it.                                             |

## 1.3 Who does what

|                                                                                                                                                                                                                                                                                                       |                                                                                                                                                                                                                                                                                                                         |                                                                                                                                                                                                                                                                                                                                                                    |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Broker administrator</b> <ul style="list-style-type: none"><li>Registers applications and issues API keys</li><li>Creates topics and subscriptions</li><li>Grants Publish / Receive permissions</li><li>Allowlists webhook hosts</li><li>Requeues dead letters; rotates keys and secrets</li></ul> | <b>Publisher (producer)</b> <ul style="list-style-type: none"><li>Keeps its API key secret</li><li>Publishes with an <code>Idempotency-Key</code></li><li>Retries on 503 or network errors with the same key</li><li>Uses an outbox so no message is lost</li><li>Sets <code>correlationId</code> for tracing</li></ul> | <b>Subscriber (consumer)</b> <ul style="list-style-type: none"><li>Receives by webhook, SignalR or pull</li><li>Processes idempotently: deduplicates on <code>messageId</code></li><li>ACKs only after the work is committed</li><li>NACKs failures; dead-letters bad messages</li><li>Verifies webhook signatures</li><li>Watches its dead-letter queue</li></ul> |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

## 1.4 Delivery guarantees

- No accepted message is lost.** The broker answers `201` only after the message and all its deliveries are committed to the database.

- **At-least-once delivery.** A delivery can arrive more than once, for example when a subscriber crashes before acknowledging, when a lease expires, or after a broker restart. **Subscribers must deduplicate.**
- **One holder at a time.** A delivery is leased to one consumer at a time. A settle with a stale lock token is refused with 410.
- **Isolation.** A failing subscription never delays or changes another subscription's deliveries.
- **Ordering is not guaranteed.** Deliveries are generally handed out oldest first, but retries, parallel consumers and redeliveries mean messages can be processed out of order. Design for that (see 3.4).

## 2. Getting connected

### 2.1 What to ask the administrator for

| You are...           | Ask for                                                                                                                                                                              | You will receive                                                              |
|----------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------|
| A publisher          | An application registration, and Publish permission on the topic(s) you send to (or a new topic, with its default time-to-live).                                                     | Your API key (shown once) and the topic name.                                 |
| A webhook subscriber | A subscription on the topic, in Webhook mode, with your HTTPS endpoint URL. Your endpoint's host must be on the broker's webhook allowlist. Say which retry settings you need (9.1). | Your API key, the subscription ID and the <b>webhook secret</b> (shown once). |
| A SignalR subscriber | A subscription in SignalR mode.                                                                                                                                                      | Your API key and the subscription ID.                                         |
| A pull subscriber    | A subscription in Pull mode.                                                                                                                                                         | Your API key and the subscription ID.                                         |

**Secrets are shown once.** The API key and the webhook secret are displayed only when they are created. Store them in your secret store (Key Vault, environment configuration). Never put them in source control, log them, or send them in URLs. The only exception is the SignalR `access_token`, which the broker redacts from its logs.

### 2.2 What the administrator does

These steps are for the broker administrator, using an Admin API key ( `$h` holds the `Authorization` header). They are listed so both sides know what to expect.

```
# 1. Register the application and issue a key (the key is returned once)
POST /api/v1/admin/applications      {"name":"billing-service"}
POST /api/v1/admin/applications/{appId}/keys {}          # optional {"expiresAt":"2027-10-01T00:00:00Z"}

# 2. Publisher: create the topic (once) and grant Publish
POST /api/v1/topics                  {"name":"payments","defaultTtlSeconds":86400}
POST /api/v1/admin/applications/{appId}/permissions
  {"resourceType":"Topic","resourceId":"{topicId}","permission":"Publish"}

# 3. Webhook subscriber: allowlist the receiving host (once per host)
POST /api/v1/admin/webhook-hosts     {"host":"billing.internal"}

# 4. Subscriber: create the subscription. Its owner gets Receive automatically.
POST /api/v1/topics/{topicId}/subscriptions
  {"name":"billing-invoices","ownerAppId":"{appId}","deliveryMode":"Webhook",
   "webhookUrl":"https://billing.internal/webhooks/payments",
   "maxAttempts":5,"lockDurationSeconds":60,"webhookTimeoutSeconds":30}
→ the response includes "webhookSecret" (Webhook mode only, shown once)
```

Names of topics, subscriptions and applications use letters, digits, `.`, `_` and `-`, up to 100 characters, and are unique ignoring case.

### 2.3 Authenticating your calls

Every REST call carries the API key in the `Authorization` header with the `ApiKey` scheme:

```
Authorization: ApiKey mbk_k3x9q2...
```

The SignalR hub takes the same key as an access token (see chapter 6). Calls with no key, or with an unknown, deactivated or expired key, get `401`. Calls outside your permissions get `403`.

| Permission | On    | Lets you                |
|------------|-------|-------------------------|
| Publish    | Topic | Publish messages to it. |

|         |              |                                                                                                                                                |
|---------|--------------|------------------------------------------------------------------------------------------------------------------------------------------------|
| Receive | Subscription | Pull, ACK, NACK and renew its deliveries; join it on the SignalR hub; read its dead-letter queue. The owning application has it automatically. |
| Manage  | Subscription | Everything Receive allows.                                                                                                                     |

**Key rotation.** When the administrator gives you a new key, deploy it, check that your calls succeed, then tell the administrator to deactivate the old key. Both keys work in between.

### Using the .NET client library

.NET applications can use the `MessageBroker.Contracts` package. It includes the request and response models, the typed `BrokerClient`, the `SignalRDeliveryListener` and the webhook signature verifier.

```
// Program.cs: registers BrokerClient as a typed HttpClient with the base address and the key
builder.Services.AddBrokerClient(o =>
{
    o.BaseAddress = new Uri(builder.Configuration["Broker:Url"]!); // e.g. https://broker.internal/
    o.ApiKey      = builder.Configuration["Broker:ApiKey"]!;      // from your secret store
});
```

Other languages call the REST API directly. Every endpoint takes and returns JSON with camelCase names.

## 3. Publisher guide

### 3.1 Publishing a message

```
POST /api/v1/topics/{topicName}/messages
Authorization: ApiKey <your key>
Idempotency-Key: 9f1c2d7e-order-48213-paid ← recommended
Content-Type: application/json

{
  "messageType": "PaymentProcessed.v1",           ← required, ≤ 200 chars
  "correlationId": "ORDER-48213",                 ← optional, ≤ 100 chars (defaults to the message ID)
  "ttlSeconds": 3600,                             ← optional; otherwise the topic default
  "properties": { "tenant": "in-01", "region": "south" }, ← optional, string values only, ≤ 4 KB
  "payload": { "paymentId": "PAY-12345", "amount": 1500.00, "currency": "INR" } ← required JSON, ≤ 256 KB
}
```

| Response                       | Meaning                                                                                                                                                | What you do                                                |
|--------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------|
| 201 Created                    | Stored and committed. The body is { "messageId": "...", "deliveryCount": 3 }, where deliveryCount is the number of subscriptions that will receive it. | Done. Record the messageId if you want to trace it.        |
| 200 OK                         | Duplicate: a message with this Idempotency-Key was already accepted. The body holds the <b>original</b> message ID.                                    | Treat as success.                                          |
| 400                            | Validation failed. The problem details list each field and why.                                                                                        | Fix the message. Do not retry unchanged.                   |
| 401 / 403                      | Bad key, or no Publish permission on this topic.                                                                                                       | Check the configuration or ask the administrator.          |
| 404                            | The topic does not exist.                                                                                                                              | Check the topic name.                                      |
| 413                            | Payload over 256 KB.                                                                                                                                   | Send a reference (see 3.4).                                |
| 503 or network error / timeout | The broker or its database is temporarily unavailable. The message may or may not have been stored.                                                    | <b>Retry with the same Idempotency-Key</b> , with backoff. |

A topic with no subscriptions still accepts messages ( deliveryCount 0). They are stored for tracing but nobody receives them, and subscriptions created later do not receive earlier messages.

#### With the .NET client

```
public sealed class PaymentEvents(BrokerClient broker)
{
    public async Task PublishPaidAsync(Payment p, CancellationToken ct)
    {
        var result = await broker.PublishAsync("payments", new PublishRequest
        {
            MessageType = "PaymentProcessed.v1",
            CorrelationId = p.OrderId,
            Properties = JsonSerializer.SerializeToElement(new { tenant = p.Tenant }),
            Payload = JsonSerializer.SerializeToElement(new { p.PaymentId, p.Amount, p.Currency }),
        },
        idempotencyKey: $"payment-{p.PaymentId}-processed", ct: ct);

        // result.MessageId, result.DeliveryCount, result.IsDuplicate (true for a 200 replay)
    }
}
```

Errors are raised as BrokerApiException, with StatusCode, ProblemType, Detail and IsTransient (true for 503 and timeouts).

#### With PowerShell or curl

```
curl -X POST https://broker.internal/api/v1/topics/payments/messages \
-H "Authorization: ApiKey $BROKER_KEY" -H "Idempotency-Key: payment-PAY-12345-processed" \
-H "Content-Type: application/json" \
-d '{"messageType": "PaymentProcessed.v1", "correlationId": "ORDER-48213", "payload": {"paymentId": "PAY-12345", "amount": 1500}}'
```

### 3.2 Idempotency and retries

A publish can fail after the broker has stored the message but before you get the answer, for example on a timeout. If you retry without an `Idempotency-Key`, subscribers receive the message twice. With a key, the retry returns `200` and the original ID, and nothing is duplicated.

- Use a key that identifies the **business event**, not the HTTP attempt: `payment-PAY-12345-processed`, or a GUID stored with the event in your outbox. At most 100 characters.
- Keys are scoped to your application and the topic, so other publishers' keys never clash with yours.
- Retry only on `503`, `408`, timeouts and connection errors, with exponential backoff (for example 1, 2, 4, 8 s... up to a minute). Never retry `4xx` errors unchanged.

### 3.3 The outbox pattern (recommended)

If your service writes to its own database and then publishes, a crash between the two steps loses the message. To prevent that, write the message to an **outbox table** in the **same transaction** as your business change. A background worker then sends outbox rows to the broker.

```
BEGIN TRAN
  UPDATE Payments SET Status = 'Paid' WHERE PaymentId = @id;
  INSERT Outbox (OutboxId, Topic, MessageType, CorrelationId, Payload, IdempotencyKey, CreatedAt)
    VALUES (NEWID(), 'payments', 'PaymentProcessed.v1', @orderId, @json, @key, SYSUTCDATETIME());
COMMIT

-- Worker loop (every few seconds):
-- read unsent rows oldest first → POST with the row's IdempotencyKey
-- 201 or 200 → mark the row sent
-- 503 / network error → leave it and try again next pass (same key, so no duplicates)
-- 400/403/404/413 → mark it failed and alert: retrying will not help
```

The broker's `samples/SamplePublisher` project implements exactly this. While the broker is down, rows wait in the outbox and are sent once it is back, with no duplicates.

### 3.4 Message design

- **Version the type** (`PaymentProcessed.v1`). For a breaking change, publish a new type (`.v2`), possibly both for a while, so subscribers can move at their own pace.
- **Make the payload self-contained** but small: identifiers, the facts that changed, and timestamps. For large documents, put the document in shared storage and send its reference. The payload limit is 256 KB.
- **Put routing and diagnostic metadata in properties** (tenant, region, source). Values must be strings, at most 4 KB in total.
- **Do not put secrets or unnecessary personal data in messages**. Payloads are stored in the broker database, kept for 14 days after completion (90 days in the DLQ), and visible to the subscribers of the topic.
- **Expect out-of-order processing**. Include a version number or an "as of" timestamp so subscribers can ignore stale updates.
- **Use `ttlSeconds`** for messages that become useless after a while, such as a one-time code or a price quote. An undelivered message past its TTL is dead-lettered as `Expired` instead of being delivered late.
- **Set `correlationId`** to the ID of the business flow (an order or request ID) and pass it on in any message you publish as a result. The whole flow can then be traced with one query (3.5).

### 3.5 Tracking what you sent

```
GET /api/v1/messages/{messageId}           # the publisher that sent it, or an Admin
GET /api/v1/messages?correlationId=ORDER-48213  # Admin only: every message in the flow
```

The message view shows the overall status (InProgress, Completed, PartiallyDeadLettered or DeadLettered). For each subscription it shows the delivery state and every attempt, with its outcome, HTTP status, duration and error. With .NET, call `broker.GetMessageAsync(messageId)`.

### 3.6 Publisher checklist

- ☐ API key loaded from a secret store; never logged.
- ☐ Every publish has an Idempotency-Key that identifies the business event.
- ☐ Retries only on 503, timeouts and network errors, with backoff and the same key.
- ☐ Outbox table written in the same transaction as the business change.
- ☐ `messageType` versioned; payload under 256 KB with no secrets; `correlationId` set.
- ☐ Alerting on outbox rows that are stuck or failed.

## 4. Subscriber guide

### 4.1 Choosing a delivery mode

| Mode           | How you receive                                                                                                                | Best for                                                                                             | You need                                                                   |
|----------------|--------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------|
| <b>Webhook</b> | The broker POSTs each delivery to your HTTPS endpoint. Your HTTP response settles it.                                          | Web services that already expose HTTPS endpoints; low latency without a long-lived connection.       | A reachable HTTPS endpoint on an allowlisted host; signature verification. |
| <b>SignalR</b> | Your app keeps a connection to the broker's hub and gets a <code>Deliver</code> event, then ACKs or NACKs over the connection. | Apps that cannot expose an endpoint (behind NAT, desktop apps, Blazor Server); real-time UI updates. | The .NET <code>SignalRDeliveryListener</code> or any SignalR client.       |
| <b>Pull</b>    | Your app asks for deliveries ( <code>/receive</code> , optionally long-polling) and ACKs over REST.                            | Batch jobs, workers that control their own pace, and scaling out by running more workers.            | A polling loop.                                                            |

### 4.2 Rules every subscriber follows

1. **Deduplicate on `message.messageId`**. Delivery is at-least-once. Record processed message IDs in your own database, in the **same transaction** as the work they cause, for example in a `ProcessedMessages(MessageId PRIMARY KEY)` table. When a message ID is already there, skip the work but still ACK.
2. **ACK only after your work is committed**. If you ACK first and then crash, the message is gone. If you commit first and then crash before the ACK, you get the message again, and rule 1 handles that.
3. **Settle before `lockedUntil`**. After the lease ends, the broker treats the attempt as failed and redelivers, and your late ACK gets `410`. For long work, **renew** the lease.
4. **NACK to retry, dead-letter to give up**. NACK a temporary failure (database busy, a downstream service down), and the broker retries with backoff. NACK with `deadLetter: true` a message that can never succeed (invalid content, an unknown type). Retrying that would only waste attempts.
5. **Ignore message types you don't handle**, or reject them on purpose. Topics can carry several types, and new versions appear over time. Decide which behaviour you want and make it explicit.
6. **Never log payloads or keys**. Log `messageId`, `correlationId`, `deliveryId` and `attempt` instead.
7. **Watch your dead-letter queue** (chapter 8). A growing DLQ means something is wrong.

```
// Deduplication: one transaction around the check, the work and the marker
await using var tx = await db.BeginTransactionAsync(ct);
if (await db.ExecuteScalarAsync<int>("SELECT COUNT(*) FROM ProcessedMessages WHERE MessageId = @id", new { id }, tx) > 0)
    return DeliveryResult.Ack; // already done: ACK again, do nothing
await ApplyBusinessChangeAsync(payload, tx);
await db.ExecuteNonQuery("INSERT ProcessedMessages (MessageId, ProcessedAt) VALUES (@id, SYSUTCDATETIME())", new { id }, tx);
await tx.CommitAsync(ct);
return DeliveryResult.Ack;
```

### 4.3 The delivery format

All three modes use the same JSON: the webhook request body, the SignalR `Deliver` event, and each item of the pull `/receive` response.

```

{
  "deliveryId": 81234,
  "lockToken": "2c9a0f4e-5d71-4b0e-9b77-0c4b9a2f1e11",
  "lockedUntil": "2026-10-08T12:31:05.120Z",
  "attempt": 1,
  "message": {
    "messageId": "01a11b48-b86f-784f-a72d-8e1bc7526202",
    "messageType": "PaymentProcessed.v1",
    "correlationId": "ORDER-48213",
    "createdAt": "2026-10-08T12:30:04.981Z",
    "properties": { "tenant": "in-01" },
    "payload": { "paymentId": "PAY-12345", "amount": 1500.00, "currency": "INR" }
  }
}

```

← use it to ACK, NACK or renew  
 ← proves you hold the lease  
 ← settle before this (UTC)  
 ← 1 on the first try, up to maxAttempts

← deduplicate on this

#### 4.4 Settling: ACK, NACK, dead-letter, renew

| Action             | Effect                                                                                                           | How (webhook · SignalR · pull)                                                                               |
|--------------------|------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------|
| <b>ACK</b>         | Delivery completed. It is never delivered again.                                                                 | Answer 2xx · return <code>DeliveryResult.Ack</code> or call <code>hub.Ack · POST /deliveries/{id}/ack</code> |
| <b>NACK</b>        | Attempt failed. Retried after backoff, or dead-lettered as <code>MaxAttemptsExceeded</code> on the last attempt. | Answer 4xx/5xx · <code>DeliveryResult.Fail(code, msg) · POST /deliveries/{id}/nack</code>                    |
| <b>Dead-letter</b> | Straight to the DLQ as <code>RejectedBySubscriber</code> , with no further attempts.                             | REST NACK with <code>deadLetter:true · Fail(code, msg, deadLetter: true) · NACK with deadLetter:true</code>  |
| <b>Renew</b>       | Extends the lease by the subscription's lock duration (counted from now).                                        | REST renew (after a 202) · <code>listener.RenewAsync(delivery) · POST /deliveries/{id}/renew</code>          |

NACK and settle bodies carry the lock token, plus optional error details that appear in the attempt history and the DLQ:

```

POST /api/v1/deliveries/81234/nack
{ "lockToken": "2c9a0f4e-...", "errorCode": "InventoryUnavailable", "errorMessage": "Stock service timed out",
  "errorDetail": "...optional longer text...", "deadLetter": false }

```

A `410 Gone` on ACK, NACK or renew means **the lease was lost**. The lease expired, someone else holds the delivery now, or it was already settled. Do **not** retry. The broker redelivers it if needed, and your deduplication makes that safe.

#### 4.5 Retries and backoff

After a failed attempt (a NACK, a failed webhook call or an expired lease), the delivery waits before the next try:

```

delay(n) = min(retryBaseDelaySeconds × 2^(n-1), retryMaxDelaySeconds) × random(0.8 ... 1.2)

defaults (base 30 s, max 900 s, maxAttempts 4): attempt 2 after ~30 s, 3 after ~60 s, 4 after ~120 s → then the DLQ

```

Ask the administrator for settings that match your recovery time. If a downstream system can be down for an hour, the retry budget should cover an hour. For example, `maxAttempts 10` with base 30 s and max 900 s spans about 1.5 hours.

## 5. Webhook subscribers

The broker sends each delivery as an `HTTPS POST` to your subscription's `webhookUrl`, with the delivery JSON (4.3) as the body. Up to `maxConcurrentDeliveries` calls (default 8) are in flight at once for your subscription.

### 5.1 Requirements for your endpoint

- `HTTPS`, on a host the administrator has added to the webhook allowlist.
- Answers within `webhookTimeoutSeconds` (default 30 s, always shorter than the lock duration). Otherwise the call counts as a `Timeout` failure.
- **Verifies the signature** before doing anything with the body (5.3).
- Does not redirect. A 3xx answer is not followed and counts as a failure.
- Is idempotent (4.2): the same delivery can arrive again.

### 5.2 Request headers

| Header                            | Content                                                                                       |
|-----------------------------------|-----------------------------------------------------------------------------------------------|
| <code>X-Broker-Delivery-Id</code> | The delivery ID (also in the body).                                                           |
| <code>X-Broker-Lock-Token</code>  | The lock token, needed to ACK later after a 202.                                              |
| <code>X-Broker-Attempt</code>     | The attempt number.                                                                           |
| <code>X-Broker-Timestamp</code>   | Unix time in seconds when the request was signed.                                             |
| <code>X-Broker-Signature</code>   | <code>sha256=&lt;hex&gt;</code> , or two comma-separated signatures during a secret rotation. |

### 5.3 Verifying the signature

The signature is **HMAC-SHA256** with your webhook secret as the key, over the string `timestamp + "." + rawBody`, written as lowercase hex with the prefix `sha256=`. To verify a request:

1. Read the **raw** request body as text, exactly as received, before any JSON parsing.
2. Reject the request if `X-Broker-Timestamp` is more than 5 minutes from your clock. This stops replays.
3. Compute `"sha256=" + hex(HMAC_SHA256(secret, timestamp + "." + body))`.
4. Compare it in constant time with **each** comma-separated value in `X-Broker-Signature`, and accept if any one matches.
5. If none matches, answer `401` and do not process the request.

```
// .NET: the Contracts package does all of the above
app.MapPost("/webhooks/payments", async (HttpRequest request, IPaymentHandler handler) =>
{
    using var reader = new StreamReader(request.Body);
    var body = await reader.ReadToEndAsync();
    if (!WebhookSignature.Verify(
        request.Headers[WebhookHeaders.Signature], request.Headers[WebhookHeaders.Timestamp],
        body, DateTimeOffset.UtcNow, secrets)) // secrets: current (and old, during rotation)
        return Results.Unauthorized();

    var delivery = JsonSerializer.Deserialize<Delivery>(body, JsonSerializerOptions.Web)!;
    return await handler.HandleAsync(delivery) switch
    {
        Outcome.Done => Results.Ok(), // ACK (also for an already-processed duplicate)
        Outcome.TryLater => Results.StatusCode(503), // NACK: retried with backoff
        Outcome.Invalid => Results.UnprocessableEntity(), // NACK; or REST NACK with deadLetter (5.4)
    };
});
```

```
# Python equivalent of the check
import hmac, hashlib, time
def verify(sig_header, ts, raw_body: bytes, secrets):
    if abs(time.time() - int(ts)) > 300: return False
    msg = ts.encode() + b"." + raw_body
    expected = ["sha256=" + hmac.new(s.encode(), msg, hashlib.sha256).hexdigest() for s in secrets]
    return any(hmac.compare_digest(e, p.strip()) for e in expected for p in sig_header.split(","))
```

## 5.4 How your response is treated

| Your response                        | Broker treats it as                                                                                                                                                                                                                                |
|--------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 200 – 299 (except 202)               | <b>ACK</b> : the delivery is completed.                                                                                                                                                                                                            |
| 202 Accepted                         | <b>Hold</b> : you have taken the work and will settle it yourself through REST, using the lock token from the headers, before <code>lockedUntil</code> . Renew if you need longer. With no ACK, the lease expires and the delivery is redelivered. |
| 3xx                                  | NACK ( <code>Http3xx</code> ). Redirects are not followed.                                                                                                                                                                                         |
| 4xx, 5xx                             | NACK ( <code>Http&lt;status&gt;</code> ), retried with backoff. Up to 2,000 characters of your response body are kept in the attempt history, so return a short, useful error and no sensitive data.                                               |
| No answer within the timeout         | NACK ( <code>Timeout</code> ).                                                                                                                                                                                                                     |
| Connection refused, DNS or TLS error | NACK ( <code>ConnectionError</code> ).                                                                                                                                                                                                             |

An HTTP response cannot dead-letter a message by itself. To reject one permanently, answer `202` and then send `POST /api/v1/deliveries/{id}/nack` with `"deadLetter": true` and the lock token from the headers. Your application's API key needs Receive on the subscription, which the owner has.

```
// 202 + asynchronous settle (long work, or permanent rejection)
var deliveryId = long.Parse(request.Headers[WebhookHeaders.DeliveryId!]);
var lockToken = Guid.Parse(request.Headers[WebhookHeaders.LockToken!]);
_ = queue.EnqueueAsync(delivery); // your worker later calls:
// POST /api/v1/deliveries/{deliveryId}/ack {"lockToken":"..."} or /nack {"lockToken":"...", "deadLetter":true,...}
return Results.Accepted();
```

## 5.5 Circuit breaker

If your endpoint fails 5 times in a row (any non-2xx status, timeout or connection error), the broker **opens a circuit** for your subscription and stops calling for 60 seconds. No attempts are used up meanwhile, and messages simply wait. It then makes one trial call. If the trial succeeds, delivery resumes at full speed. This protects your service while it recovers, and it never affects other subscriptions.

## 5.6 Webhook secret rotation

When the administrator rotates your secret (`POST /api/v1/subscriptions/{id}/webhook-secret`), you receive the new secret. For the next **24 hours**, every request carries two signatures, made with the new and the old secret. Configure your endpoint to accept both secrets, deploy the new one, then remove the old one before the 24 hours are up.

## 6. SignalR subscribers

Your application opens a SignalR connection to the broker's hub at `/hubs/deliveries` and **subscribes** to one or more subscriptions. The broker pushes each delivery to **exactly one** connected client of that subscription. With several clients (several instances of your app), deliveries are shared round-robin. When no client is connected, nothing is handed out, and messages wait in the broker.

### 6.1 Using the .NET listener (recommended)

`SignalRDeliveryListener` handles connecting, authentication, subscribing, ACK and NACK from your handler's result, and reconnecting forever after a network drop or a broker restart (it retries at 0, 2, 5 and 10 s, then every 30 s, and subscribes again by itself).

```
await using var listener = new SignalRDeliveryListener(
    new Uri("https://broker.internal/"), apiKey,
    async (delivery, ct) =>
    {
        if (delivery.Message.MessageType != "PaymentProcessed.v1")
            return DeliveryResult.Fail("UnknownType", delivery.Message.MessageType, deadLetter: true);
        try
        {
            await ProcessIdempotentlyAsync(delivery, ct);    // dedupe on delivery.Message.MessageId
            return DeliveryResult.Ack;
        }
        catch (DbException ex)
        {
            return DeliveryResult.Fail("DatabaseBusy", ex.Message); // retried with backoff
        }
    });

listener.SettlementFailed += (d, ex) => logger.LogWarning("Could not settle {DeliveryId}: {Error}", d.DeliveryId,
ex.Message);
await listener.StartAsync([subscriptionId], stoppingToken);
```

- An exception thrown by your handler is treated as a NACK (retry).
- Handlers run concurrently. The broker never sends more than `maxConcurrentDeliveries` unsettled deliveries (default 8) to your subscription.
- For work that may outlast the lease, call `await listener.RenewAsync(delivery)`. It returns the new `lockedUntil`.
- Host it in a `BackgroundService`. The broker samples include `SignalRSubscriberWorker`, which also retries the first connection while the broker is starting.
- In Blazor Server, raise an event from the handler and call `InvokeAsync(StateHasChanged)` in components. `samples/BlazorSubscriber` shows toast notifications built this way.

### 6.2 Using any SignalR client

| Item               | Detail                                                                                                                                                                                                                                                                                                         |
|--------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| URL                | <code>https://&lt;broker&gt;/hubs/deliveries</code>                                                                                                                                                                                                                                                            |
| Authentication     | Your API key as the access token. Clients send it as the <code>access_token</code> query parameter (for example the JavaScript client's <code>accessTokenFactory</code> ), and the broker redacts it from logs.                                                                                                |
| Server → client    | <code>Deliver(delivery)</code> , carrying the JSON from 4.3.                                                                                                                                                                                                                                                   |
| Client → server    | <code>Subscribe(subscriptionId)</code> , <code>Unsubscribe(subscriptionId)</code> , <code>Ack(deliveryId, lockToken)</code> , <code>Nack(deliveryId, lockToken, errorCode, errorMessage, deadLetter)</code> , and <code>Renew(deliveryId, lockToken)</code> , which returns the new <code>lockedUntil</code> . |
| Errors             | A failed call raises a hub error whose message starts with the kind: <code>Forbidden:</code> , <code>NotFound:</code> , <code>Conflict:</code> , <code>LeaseLost:</code> or <code>Validation:</code> .                                                                                                         |
| After reconnecting | <b>Call <code>Subscribe</code> again</b> for every subscription. The broker does not remember connections across a disconnect or restart.                                                                                                                                                                      |

```
// JavaScript (@microsoft/signalr)
const conn = new signalR.HubConnectionBuilder()
  .withUrl("https://broker.internal/hubs/deliveries", { accessTokenFactory: () => apiKey })
  .withAutomaticReconnect([0, 2000, 5000, 10000, 30000]).build();
conn.on("Deliver", async d => {
  try { await handle(d); await conn.invoke("Ack", d.deliveryId, d.lockToken); }
  catch (e) { await conn.invoke("Nack", d.deliveryId, d.lockToken, "HandlerError", String(e), false); }
});
conn.onreconnected(() => conn.invoke("Subscribe", subscriptionId));
await conn.start(); await conn.invoke("Subscribe", subscriptionId);
```

**If your client disconnects** while holding deliveries, they are redelivered after their lease expires. That can be to another instance, or to the same one after it reconnects.

## 7. Pull subscribers

Your worker asks for deliveries when it is ready for them, so you control the pace completely. Run more workers to process faster, because each delivery is leased to only one of them.

### 7.1 Receiving

```
POST /api/v1/subscriptions/{subscriptionId}/receive
{ "maxMessages": 16, "waitSeconds": 20 }

200 OK → [ {delivery}, {delivery}, ... ]      (an empty list when nothing arrived in time)
```

- `maxMessages` is 1–32 (default 1). `waitSeconds` is 0–30 (default 0, which returns at once). Values out of range are clamped.
- With `waitSeconds` above 0 the call **long-polls**: it returns as soon as a message is published, or empty after the wait. Use 10–30 s so an idle worker is quiet and still reacts at once.
- Deliveries come oldest first. Paused subscriptions, deliveries that are not due yet (backoff) and expired deliveries are skipped.
- Every delivery in the batch has its own lease. Settle each one separately.

### 7.2 Settling

```
POST /api/v1/deliveries/{deliveryId}/ack    {"lockToken":"..."} → 204
POST /api/v1/deliveries/{deliveryId}/nack   {"lockToken":"...", "errorCode":"...", "errorMessage":"...",
                                           "deadLetter":false} → 204
POST /api/v1/deliveries/{deliveryId}/renew  {"lockToken":"..."} → 204
                                           (any of them → 410 when the lease was lost)
```

### 7.3 A worker loop (.NET)

```
protected override async Task ExecuteAsync(CancellationToken stop)
{
    while (!stop.IsCancellationRequested)
    {
        IReadOnlyList<Delivery> batch;
        try { batch = await broker.ReceiveAsync(subscriptionId, maxMessages: 16, waitSeconds: 20, stop); }
        catch (BrokerApiException ex) when (ex.IsTransient) { await Task.Delay(5000, stop); continue; }

        foreach (var delivery in batch)
        {
            try
            {
                await ProcessIdempotentlyAsync(delivery, stop);           // dedupe on Message.MessageId
                await broker.AckAsync(delivery, stop);
            }
            catch (InvalidMessageException ex)
            {
                await broker.NackAsync(delivery, "Invalid", ex.Message, deadLetter: true, stop);
            }
            catch (BrokerApiException ex) when (ex.IsLeaseLost)
            {
                // lease expired while working: it will be redelivered; deduplication makes that safe
            }
            catch (Exception ex)
            {
                await broker.NackAsync(delivery, "ProcessingFailed", ex.Message, ct: stop);
            }
        }
    }
}
```

**Size the batch to the lease.** All deliveries in a batch must be settled before `lockedUntil`. If one batch takes longer than the lock duration (default 60 s), ask for fewer messages, process them in parallel, or renew.

## 8. Dead letters and troubleshooting

### 8.1 Reading your dead-letter queue

```
GET /api/v1/subscriptions/{subscriptionId}/deadletters?pageSize=50
GET /api/v1/subscriptions/{subscriptionId}/deadletters?pageSize=50&before={nextBefore} # next page
GET .../deadletters?includeRequeued=true # history too
```

Entries come newest first. Each one shows the `reason`, the attempt count, `lastError`, the first and last failure times, and the full message. Owners (Receive permission) and Admins can read it.

| Reason               | Cause                                                              | Typical fix                                                                                       |
|----------------------|--------------------------------------------------------------------|---------------------------------------------------------------------------------------------------|
| MaxAttemptsExceeded  | Every attempt failed (NACK, error status, timeout, expired lease). | Fix the bug or outage, then ask for a requeue.                                                    |
| RejectedBySubscriber | You NACKed it with <code>deadLetter: true</code> .                 | Requeue only after your code changed to accept it, or ask the publisher to correct and republish. |
| Expired              | Its time-to-live passed before it could be delivered.              | Usually leave it. A requeue would deliver stale data.                                             |

### 8.2 Getting a message requeued

Only an administrator can requeue (POST `/api/v1/deadletters/{deliveryId}/requeue`). Send them the subscription, the delivery IDs and confirmation that the cause is fixed. A requeued delivery starts with a fresh retry budget (its attempt number starts at 1 again), while the message history keeps every earlier attempt. In Phase 1 entries are requeued one at a time.

### 8.3 Errors and what to do

Errors use RFC 9457 problem details (`application/problem+json`) with a `type`, a `title` and a `detail`. Validation errors also list each failing field.

| Status | Problem type (urn:message-broker:problem:...) | Meaning                                                                                             | Action                                     |
|--------|-----------------------------------------------|-----------------------------------------------------------------------------------------------------|--------------------------------------------|
| 400    | validation                                    | The request is invalid; see the field errors.                                                       | Fix the request.                           |
| 401    | unauthorized                                  | Missing, unknown, deactivated or expired key.                                                       | Check the key and its expiry.              |
| 403    | forbidden                                     | Valid key, but no permission for this topic or subscription.                                        | Ask the administrator.                     |
| 404    | not-found                                     | Unknown topic, subscription, delivery or message.                                                   | Check the name or ID.                      |
| 409    | conflict                                      | The state does not allow it (for example a duplicate name, or requeueing something not in the DLQ). | Read the detail.                           |
| 410    | lease-lost                                    | The lease ended, or the token is stale.                                                             | Do not retry; it will be redelivered.      |
| 413    | payload-too-large                             | Payload over 256 KB. (Properties over 4 KB give a 400.)                                             | Send a reference instead.                  |
| 503    | unavailable                                   | Temporary store problem.                                                                            | Retry with backoff (publishers: same key). |

#### Common problems

| Symptom | Likely cause |
|---------|--------------|
|---------|--------------|

|                                               |                                                                                                                                                                |
|-----------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Webhook never called                          | Subscription paused; circuit open after repeated failures; the host was removed from the allowlist; the message was published before the subscription existed. |
| Signature never verifies                      | The body was re-serialized instead of verifying the raw text; wrong or old secret; server clock more than 5 minutes off.                                       |
| Same message processed twice                  | Expected with at-least-once delivery. Add deduplication on <code>messageId</code> (4.2).                                                                       |
| ACK returns 410                               | Processing took longer than the lock duration. Renew, use smaller batches, or ask for a longer lock (up to 600 s).                                             |
| SignalR client connected but receives nothing | <code>Subscribe</code> was not called (again) after connecting; the subscription is not in SignalR mode; the key lacks <code>Receive</code> .                  |
| Messages dead-lettered as Expired             | The subscriber was down or paused longer than the message TTL.                                                                                                 |

To investigate a message, ask the publisher or an administrator to look it up with `GET /api/v1/messages/{messageId}`. It shows every attempt on every subscription, with the error codes your side returned.

## 9. Reference

### 9.1 Subscription settings

Set by the administrator when creating or changing the subscription ( `PATCH /api/v1/subscriptions/{id}` ). Settings left out take the broker defaults.

| Setting                              | Default | Allowed                      | Notes                                                                                       |
|--------------------------------------|---------|------------------------------|---------------------------------------------------------------------------------------------|
| <code>deliveryMode</code>            | —       | Webhook, SignalR, Pull       | Cannot be changed later; create a new subscription instead.                                 |
| <code>webhookUrl</code>              | —       | HTTPS, allowlisted host      | Webhook mode only.                                                                          |
| <code>webhookTimeoutSeconds</code>   | 30      | $\geq 1$ , $<$ lock duration | Webhook mode only.                                                                          |
| <code>lockDurationSeconds</code>     | 60      | 1–600                        | How long you have to settle a delivery.                                                     |
| <code>maxAttempts</code>             | 4       | 1–100                        | Attempts before the DLQ.                                                                    |
| <code>retryBaseDelaySeconds</code>   | 30      | $\geq 1$                     | First retry delay (4.5).                                                                    |
| <code>retryMaxDelaySeconds</code>    | 900     | base – 86,400                | Cap on the retry delay.                                                                     |
| <code>maxConcurrentDeliveries</code> | 8       | 1–256                        | Webhook calls in flight, or unsettled SignalR deliveries.                                   |
| <code>ttlSeconds</code>              | none    | $> 0$                        | Expiry for this subscription's deliveries. The earlier of this and the message TTL applies. |
| <code>status</code>                  | Active  | Active, Paused               | While Paused, messages queue but nothing is delivered, and TTLs keep running.               |

### 9.2 Limits

| Item                                                                                 | Limit                                                                                 |
|--------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------|
| Payload                                                                              | 256 KB of JSON                                                                        |
| Properties                                                                           | JSON object of string values, 4 KB                                                    |
| <code>messageType</code> / <code>correlationId</code> / <code>Idempotency-Key</code> | 200 / 100 / 100 characters                                                            |
| Names (topic, subscription, application)                                             | [A-Za-z0-9._-], 1–100 characters, unique ignoring case                                |
| Pull batch / long-poll wait                                                          | 32 messages / 30 seconds                                                              |
| Active API keys per application                                                      | 2                                                                                     |
| Webhook signature timestamp tolerance                                                | 5 minutes                                                                             |
| Retention                                                                            | Completed work 14 days; dead letters 90 days                                          |
| Target throughput (Phase 1)                                                          | 100 messages/s sustained; publish p95 under 100 ms; delivery under 1 s at normal load |

### 9.3 Endpoints

| Endpoint                                             | Purpose            | Who                  |
|------------------------------------------------------|--------------------|----------------------|
| <code>POST /api/v1/topics/{name}/messages</code>     | Publish            | Publish permission   |
| <code>GET /api/v1/messages/{messageId}</code>        | Message history    | Its publisher, Admin |
| <code>GET /api/v1/messages?correlationId=</code>     | Messages of a flow | Admin                |
| <code>POST /api/v1/subscriptions/{id}/receive</code> | Pull deliveries    | Receive              |

|                                                 |                                                |                |
|-------------------------------------------------|------------------------------------------------|----------------|
| POST /api/v1/deliveries/{id}/ack   nack   renew | Settle or extend                               | Receive        |
| GET /api/v1/subscriptions/{id}/deadletters      | Read the DLQ                                   | Receive, Admin |
| POST /api/v1/deadletters/{deliveryId}/requeue   | Requeue                                        | Admin          |
| /hubs/deliveries (SignalR)                      | Push channel                                   | Receive        |
| /api/v1/topics..., /api/v1/subscriptions...     | Topics and subscriptions                       | Admin          |
| /api/v1/admin/...                               | Applications, keys, permissions, webhook hosts | Admin          |
| GET /health/live, /health/ready                 | Liveness and readiness                         | Anyone         |

The OpenAPI description is served by the broker ( /openapi/v1.json ) for code generation in other languages.

## 9.4 Subscriber checklist

- ☐ API key (and webhook secret) loaded from a secret store; never logged.
- ☐ Deduplication on `messageId` in the same transaction as the work.
- ☐ ACK only after commit; NACK temporary failures; dead-letter invalid messages.
- ☐ Work finishes well inside the lock duration, or renews the lease; 410 handled without retrying.
- ☐ Webhook: HTTPS on an allowlisted host; raw-body signature check that accepts two secrets; answers within the timeout; no redirects.
- ☐ SignalR: Subscribe again after every reconnect (automatic with `SignalRDeliveryListener`).
- ☐ Pull: long-poll; batch size matched to the lease; transient errors retried with backoff.
- ☐ Unknown message types handled on purpose (ignore with ACK, or dead-letter).
- ☐ Retry settings agreed with the administrator to cover your expected outages.
- ☐ Dead-letter queue monitored, with an owner who acts on it.

## 9.5 Glossary

|                           |                                                                                                                       |
|---------------------------|-----------------------------------------------------------------------------------------------------------------------|
| <b>ACK / NACK</b>         | Acknowledge success, or report failure, for one delivery.                                                             |
| <b>At-least-once</b>      | Every message is delivered, possibly more than once; the receiver deduplicates.                                       |
| <b>Backoff</b>            | The growing wait between retries.                                                                                     |
| <b>Circuit breaker</b>    | A pause in webhook calls after repeated failures, to protect a struggling endpoint.                                   |
| <b>Idempotency-Key</b>    | A publisher-chosen key that makes repeating a publish safe.                                                           |
| <b>Lease / lock token</b> | Temporary exclusive ownership of a delivery, and the token proving it.                                                |
| <b>Outbox</b>             | A table in the publisher's database holding messages to send, written in the same transaction as the business change. |
| <b>TTL</b>                | Time-to-live: how long a message stays worth delivering.                                                              |